

CONNECTIONS BETWEEN SCHEDULE-FREE OPTIMIZERS, AdEMAMix, AND ACCELERATED SGD VARIANTS

Anonymous authors
Anonymous submission

ABSTRACT

Recent deep learning optimizers such as Schedule-Free methods, AdEMAMix, MARS, and Lion broaden the design space of practical training algorithms by altering how momentum enters the update. In parallel, theoretical work on stochastic gradient descent (SGD) has shown that acceleration in noise-dominated regimes can be obtained by decoupling the momentum coefficient from the weight assigned to the current gradient. This paper connects these perspectives and shows that several modern optimizer designs can be interpreted through a common accelerated-SGD lens. We support the theoretical connections with controlled preliminary experiments in a 150m-parameter language modeling setting. Within this setting, AdEMAMix, whose update most closely matches accelerated SGD structure among the tested optimizers, achieves the strongest performance. Motivated by this interpretation, we introduce Simplified-AdEMAMix, a modification that preserves AdEMAMix-level performance in both small- and large-batch configurations while removing the need for two separate momentum terms. The code for Simplified-AdEMAMix will be released upon acceptance.

1 INTRODUCTION

Recent work in deep learning optimization has produced a range of optimizer designs, including Lion (Chen et al., 2023), ScheduleFreeSGD/AdamW (Defazio et al., 2024), and AdEMAMix (Pagliardini et al., 2024). Although these methods were introduced for different practical and conceptual reasons, they share a structural theme that extends beyond any single training task: each changes the way momentum is formed, weighted, or combined with the current gradient.

A complementary theoretical literature studies when gradient methods can be accelerated under stochastic, noise-dominated conditions. Classical momentum schemes such as heavy-ball and Nesterov momentum can accelerate deterministic gradient descent, especially for quadratic objectives, but they do not by themselves accelerate SGD (Jain et al., 2018; Liu & Belkin, 2020). This gap has motivated alternative stochastic momentum mechanisms designed to provide acceleration despite gradient noise (Jain et al., 2018; Vaswani et al., 2019; Liu & Belkin, 2020; Gupta et al., 2023). Across these accelerated SGD methods, a unifying pattern is that the momentum coefficient is decoupled from the direct weight placed on the current gradient in the update.

Our main contribution is to make this common structure explicit across the theoretical and practical lines of work. We show that Schedule-Free SGD is mathematically equivalent to accelerated SGD followed by weight averaging. We further interpret optimizers such as Lion, Schedule-Free AdamW, and AdEMAMix as combining preconditioning or nonlinear update transformations with accelerated-SGD-style momentum. Although parts of this relationship have appeared in prior discussions (Defazio, 2021), we are not aware of a formal treatment that places these recently proposed optimizers within the same accelerated SGD framework, making the comparison useful across optimizer families rather than only for one named method.

We evaluate the implications of this framework in controlled language modeling experiments using a 150m-parameter decoder-only transformer. The small-batch experiments train on 15b tokens with a 32k-token batch size, providing one representative regime in which stochastic gradient noise is expected to be prominent. Consistent with the theoretical interpretation, Schedule-Free AdamW behaves similarly to accelerated SGD-based AdamW (Algorithm 3), and accelerated variants provide modest gains in the small-batch regime. We also test a large-batch configuration and find that these gains diminish, matching the broader expectation that acceleration mechanisms designed for noisy SGD are less beneficial when the effective noise level is reduced.

The contributions are as follows:

1. We establish precise theoretical connections between accelerated SGD and recently proposed optimizers, including Schedule-Free SGD and AdEMAMix.
2. We empirically examine these connections on a 150-million-parameter decoder-only transformer by comparing AdamW, Schedule-Free AdamW, AdEMAMix, and MARS in the same language modeling setup. In these experiments, AdEMAMix, the tested method most closely aligned with accelerated SGD variants, performs best.
3. We show that, as suggested by the accelerated SGD interpretation, the advantages of these methods are reduced at large batch sizes relative to Adam. In this regime, Adam with momentum scheduling can match AdEMAMix performance.
4. We find that Schedule-Free AdamW performs substantially worse than AdamW with cosine decay at high batch sizes, and we attribute this behavior to the built-in coupling between momentum and weight averaging coefficients in Schedule-Free optimizers.
5. We introduce Simplified-AdEMAMix, a variant that preserves AdEMAMix performance across the small- and large-batch regimes studied here while removing the need for two distinct momentum terms.

2 RELATED WORK

We review the literature most directly connected to our scope: accelerated SGD methods that modify stochastic momentum, and modern deep learning optimizers whose update rules exhibit related structure across practical training settings.

Jain et al. (2018) introduced an accelerated SGD variant with improved convergence rates for least-squares problems. Kidambi et al. (2018) simplified the corresponding update and showed that ordinary momentum does not provide acceleration in that setting. Later work broadened the theory to general convex and strongly convex objectives under different assumptions (Liu & Belkin, 2020; Vaswani et al., 2019; Gupta et al., 2023), giving a wider family of stochastic acceleration mechanisms rather than a single algorithmic template.

Several practical optimizers developed for deep learning also resemble these accelerated SGD variants. Lucas et al. (2019) proposed using a weighted combination of multiple momentum terms with different coefficients to form the update. Ma & Yarats (2019) designed an optimizer explicitly motivated by the framework of Jain et al. (2018). More recently, Chen et al. (2023) introduced an optimizer found through genetic search that, like accelerated SGD variants, separates the gradient’s update weight from the momentum coefficient. Pagliardini et al. (2024) further expanded this design pattern by combining two momentum scales in the final update.

3 BACKGROUND

3.1 MOMENTUM

Momentum is a standard mechanism for shaping gradient descent dynamics across optimization problems, with especially clear acceleration behavior in deterministic settings. For weights w_t and momentum coefficient β , the classical update is:

$$m_t = \beta m_{t-1} + \nabla f(w_t); \quad w_t = w_{t-1} - \eta m_t$$

3.2 WEIGHT AVERAGING

Weight averaging is commonly used in stochastic optimization to make the returned parameters less sensitive to noisy iterates. Rather than returning only the final iterate w_T , the method forms a weighted iterate average $\bar{w}_T$, with averaging weights denoted by γ_t :

$$\bar{w}_T = (1 - \gamma_T)\bar{w}_{T-1} + \gamma_T w_T$$

Throughout this paper, the weight averaging schemes we consider use coefficients γ_t of the form:

$$\gamma_t \approx 1 - \frac{1}{\delta t}$$

for some constant $0 \leq \delta \leq 1$.

3.3 ACCELERATED SGD

We use the following generalized form as a shared notation for a broad class of accelerated SGD methods:

$$m_t = \beta_{a,t} m_{t-1} + g_t, \quad w_{t+1} = w_t - \eta_{a,t} m_t - \alpha_{a,t} g_t \quad (1)$$

Here, $\beta_{a,t}, \alpha_{a,t}, \eta_{a,t}$ are scalar coefficients that may vary with time, and g_t is the stochastic gradient evaluated at w_t . The subscript ‘a’ marks coefficients associated with this accelerated SGD parameterization.

Setting $\alpha_{a,t} = 0$ recovers standard SGD with momentum, while allowing $\alpha_{a,t}$ to be nonzero captures the broader class of updates that decouple the current-gradient weight from the momentum recursion. As noted in prior work and detailed in Appendix B, accelerated SGD algorithms from Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023) can all be written in this form.

4 CONNECTIONS BETWEEN EXISTING OPTIMIZERS AND ACCELERATED SGD

This section places several existing optimizer designs, including Schedule-Free optimizers and AdEMAMix, inside the accelerated SGD viewpoint. The resulting interpretation gives a common language for comparing otherwise different update rules and also motivates a simplified AdEMAMix variant that uses a single momentum term while matching AdEMAMix performance in the small- and large-batch settings evaluated in this paper.

4.1 SCHEDULE-FREE SGD

Schedule-Free SGD (Defazio et al., 2024) is a constant-learning-rate optimizer intended to reduce reliance on explicit learning-rate schedules, a design goal that is relevant across training workloads where schedule selection is costly. Using the notation of Defazio et al. (2024), its updates are:

$$\begin{aligned} y_t &= (1 - \beta) z_t + \beta x_t \\ z_{t+1} &= z_t - \gamma g(y_t) \\ x_{t+1} &= (1 - c_{t+1}) x_t + c_{t+1} z_{t+1} \end{aligned}$$

In this notation, y_t is the point at which gradients are evaluated, whereas x_t is the parameter sequence used for evaluation.

We first rewrite the dynamics in terms of y_t and an auxiliary momentum variable m_t , defined by:

$$m_{t+1} = \frac{x_t - z_{t+1}}{\gamma}. \quad (2)$$

Expanding this definition gives the following momentum recursion:

$$m_t = \frac{x_t - z_{t+1}}{\gamma} \quad (3)$$

$$= \frac{x_t + \gamma g_t - z_t}{\gamma} \quad (4)$$

$$= \frac{x_t - z_t + \gamma g_t}{\gamma} \quad (5)$$

$$= \frac{(1 - c_t)(x_{t-1} - z_t) + \gamma g_t}{\gamma} \quad (6)$$

$$= (1 - c_t)m_{t-1} + g_t. \quad (7)$$

Thus, m_t obeys the momentum update in Equation (1) with $\beta_{a,t} = 1 - c_t$. We can then express the corresponding update for y_t :

$$y_{t+1} = (1 - \beta)z_{t+1} + \beta x_{t+1} \quad (8)$$

$$= (1 - \beta)(z_t - \gamma g_t) + \beta((1 - c_{t+1})x_t + c_{t+1}z_{t+1}) \quad (9)$$

$$= (1 - \beta)z_t + \beta x_t - (1 - \beta)\gamma g_t + \beta c_{t+1}(z_{t+1} - x_t) \quad (10)$$

$$= y_t - \gamma[\beta c_{t+1}m_t + (1 - \beta)g_t]. \quad (11)$$

Therefore, the y_t sequence follows the weight update in Equation (1) with $\eta_{a,t} = \gamma\beta c_{t+1}$ and $\alpha_{a,t} = \gamma(1 - \beta)$, taking $w_t = y_t$. Schedule-Free SGD therefore contains an accelerated SGD trajectory in its training dynamics. Since Schedule-Free SGD evaluates using x_t , we next characterize that sequence:

$$\begin{aligned} x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \\ x_{t+1} &= (1 - c_{t+1})x_t + c_2 \left(\frac{y_{t+1} - \beta x_{t+1}}{1 - \beta} \right) \\ x_{t+1}(1 - \beta + c_{t+1}\beta) &= (1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1} \\ x_{t+1} &= \frac{(1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1}}{(1 - c_{t+1})(1 - \beta) + c_{t+1}}. \end{aligned}$$

The evaluation sequence x_t is consequently a weighted average of the accelerated sequence y_t . Recursively expanding x_t shows that, for constant c_t , it is an exponential average of y_t . In this sense, Schedule-Free SGD can be viewed more generally as accelerated SGD followed by weight averaging.

Under this interpretation, the benefits of Schedule-Free SGD separate into two components:

1. Potential improvement over standard SGD with momentum through its accelerated SGD component.
2. The ability to use a constant learning rate without an explicit schedule, supported by weight averaging (specifically, tailed weight averaging; see Section 4.1.3).

Schedule-Free SGD/Adam also has two practical advantages within this family of approaches:

- It avoids additional memory devoted solely to weight averaging.
- It removes an explicit weight averaging coefficient from the hyperparameter set.

However, Section 5.1 shows that coupling the momentum and weight averaging coefficients can be problematic in the large-batch regime studied here.

4.1.1 CASE: $\beta = 0.0$

As noted in (Defazio et al., 2024), the choice $\beta = 0$ reduces Schedule-Free SGD to standard SGD with weight averaging. Because $c_t = 1/t$, the averaging begins at the start of training.

4.1.2 CASE: $\beta = 1.0$

At the other endpoint, (Defazio et al., 2024) observes that $\beta = 1$ recovers standard momentum SGD, with momentum coefficient $\beta_{a,t}$ scaling as $1 - 1/t$.

4.1.3 CASE: $\beta = 0.9$

For the default Schedule-Free SGD setting $\beta = 0.9$:

- Since c_t scales as $1/t$, the effective momentum grows as $1 - 1/t$.
- The ratio between the current-gradient weight and the momentum weight is fixed at $(1 - \beta)/(\beta c_{t+1}) \approx 0.11$.
- The induced weight averaging approximately covers the most recent 10% of iterates.

4.2 LION

Lion (Chen et al., 2023) uses the following update rule:

$$m'_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_t = \theta_{t-1} - \eta \text{sign}(m'_t)$$

$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t.$$

Lion (Chen et al., 2023) can therefore be interpreted as an accelerated-SGD-style momentum construction followed by a coordinate-wise sign transformation.

4.3 MARS

We next consider the practical MARS (Yuan et al., 2024) variant, MARS-Approx. Ignoring bias correction and clipping, its update can be written as an accelerated SGD form augmented with preconditioning:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) c_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

Here, m_t and v_t denote first- and second-order momentum quantities, and x_t denotes the model parameters. Reparameterizing with $\hat{m}_t = m_t - \gamma g_t$ yields:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ \hat{m}_t &= \beta_1 \hat{m}_{t-1} + (1 - \beta_1)(1 - \gamma) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{\hat{m}_t + \gamma g_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

This form shows that the momentum component fits the general accelerated SGD framework, while the preconditioning differs from AdamW. We evaluate this optimizer alongside the other accelerated-SGD-related methods in Section 5.

4.4 ADEMAMIX

AdEMAMix (Pagliardini et al., 2024) also shares structure with accelerated SGD-based AdamW. Rather than combining the current gradient directly with a single momentum term as in accelerated SGD, AdEMAMix maintains two momentum terms with different coefficients and uses their linear combination in the update. Algorithm 1 states the algorithm.

For analysis, we focus on the AdEMAMix variant with $\beta_1 = 0$. Pagliardini et al. (2024) reports that this version performs nearly as well as the full optimizer at small batch sizes, and our experiments in Section 5 support the same observation. With $\beta_1 = 0$, the AdEMAMix update aligns with the general accelerated SGD framework in Equation (1). Moreover, the prescribed schedules for β_3 , which controls the momentum coefficient, and α , which controls the relative contribution of the current-gradient pathway, closely resemble theoretical schedules proposed for accelerated SGD (Gupta et al., 2023).

For smooth convex optimization in stochastic settings, acceleration requires a momentum schedule of the form:

$$\beta_{a,t} = 1 - \frac{k}{t}$$

for some constant $k > 0$, as established by Gupta et al. (2023). AdEMAMix approximately follows this pattern through the schedule that increases β_3 .

There is one notational difference between the theory-style accelerated SGD update and the AdEMAMix implementation. In accelerated SGD schemes, momentum is written in the standard form:

$$m_t = \beta_{a,t} m_{t-1} + g_t$$

whereas Algorithm 1 uses the EMA-style update:

$$m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}.$$

When $\beta_{a,t}$ scales as $1 - 1/t$, the accumulated contribution of past gradients in accelerated SGD grows proportionally to t . The AdEMAMix coefficient α similarly scales proportionally to t . This shared scaling provides a broader explanation for why AdEMAMix performs well relative to the other tested optimizers in Figure 1.

At large batch sizes, however, the $\beta_1 = 0.0$ version of AdEMAMix loses performance, consistent with the observations of Pagliardini et al. (2024). The analysis of Gupta et al. (2023) suggests that the weight assigned to the current gradient should decrease as batch size increases. AdEMAMix instead keeps a fixed weight of 1 on this component, which likely explains its weaker behavior in the large-batch setting.

This motivates a simplified variant that explicitly weights the current-gradient contribution, removes the schedule for α , and maintains only one momentum term. We empirically show that this variant performs comparably to AdEMAMix in both the small- and large-batch setups studied here.

4.5 SIMPLIFIED ADEMAMIX

Guided by the accelerated SGD interpretation, we propose a simplified optimizer that avoids two separate momentum terms and does not require scheduling α . Algorithm 2 presents the method using theory-style momentum rather than the exponential moving average (EMA) convention. The final update assigns a fixed weight α to the gradient; setting $\alpha = 0$ recovers standard Adam after the corresponding transformations of η and β_1 . In Section 5, this simpler update matches AdEMAMix across the small- and large-batch configurations we evaluate.

5 EXPERIMENTS

We evaluate the optimizer connections on a language modeling task with a 150-million-parameter decoder-only transformer trained on the C4 dataset. The model uses sequence length 1024 and batch size 32, and is trained for 15 billion tokens ($\approx 5 \times$ Chinchilla), giving a small-batch regime in which stochastic gradient noise is expected to be substantial.

Algorithm 1 Single step of AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters β_3, α . Warmup parameter T_{α, β_3} , note that we usually set it to T . β_{start} is usually set to β_1 .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_3^{(t)} \leftarrow f_{\beta_3}(t, \beta_3, \beta_{\text{start}}, T_{\alpha, \beta_3})$ and $\alpha^{(t)} \leftarrow f_{\alpha}(t, \alpha, T_{\alpha, \beta_3})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the fast EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + (1 - \beta_1) g^{(t)}$
 - 6: Update the slow EMA m_2 : $m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}$
 - 7: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 8: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{\hat{m}_1^{(t)} + \alpha^{(t)} m_2^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

Algorithm 2 Single step of Simplified AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters $\alpha, \beta_{\text{start}}$. Warmup parameter T_{β_1} , note that we usually set it to T .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_1^{(t)} \leftarrow f_{\beta_1}(t, \beta_1, \beta_{\text{start}}, T_{\beta_1})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + g^{(t)}$
 - 6: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 7: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{m_1^{(t)} + \alpha g^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

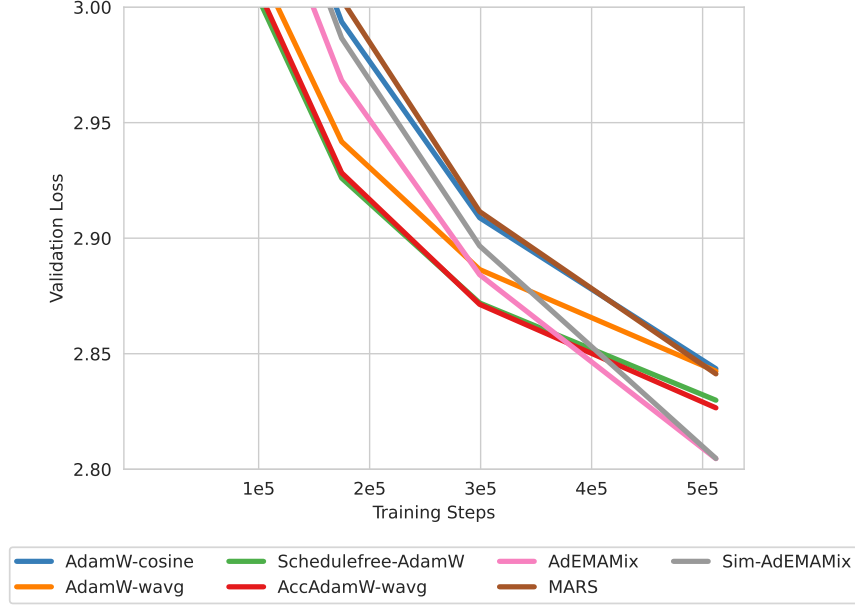


Figure 1: Best runs for the optimizer comparisons described in Section 5 on the decoder-only 150m transformer language modeling task. In this setting, AdEMAMix and simplified-AdEMAMix perform best, consistent with their close structural match to accelerated SGD variants.

Algorithm 3 Single step of accelerated SGD based Adam with weight averaging. For simplicity we ignore the initialization, other boundary effects such as bias correction, and weight decay. Hyperparameters: Learning rate η , betas $= (\beta_1, \beta_2, \beta_3)$, weight averaging coefficient δ , and epsilon ϵ .

- 1: Sample batch B_t .
- 2: $g \leftarrow -\nabla_w \phi_{B_t}(w_t)$
- 3: $v \leftarrow \beta_2 v + (1 - \beta_2)(g \odot g)$
- 4: $N \leftarrow \frac{\beta_3 m + (1 - \beta_3)g}{\sqrt{\hat{v}} + \epsilon}$
- 5: $w \leftarrow w - \eta N$
- 6: $m \leftarrow \beta_1 m + (1 - \beta_1)g$
- 7: $c = \max(1 - 1/t, 1 - 1/(\delta t))$
- 8: $w_{\text{avg}} \leftarrow c w_{\text{avg}} + (1 - c)w$

Within this shared training setup, we compare the following optimizers:

1. Standard AdamW with cosine decay
2. Standard AdamW with weight averaging
3. Schedule-Free AdamW
4. Accelerated AdamW with weight averaging (Algorithm 3)
5. MARS
6. AdEMAMix
7. Simplified-AdEMAMix

The corresponding hyperparameter sweeps are listed in Section A.

As shown in Figure 1, Schedule-Free AdamW and Accelerated AdamW with tailed weight averaging perform similarly, supporting the proposed equivalence. Both methods outperform AdamW with cosine decay and AdamW with

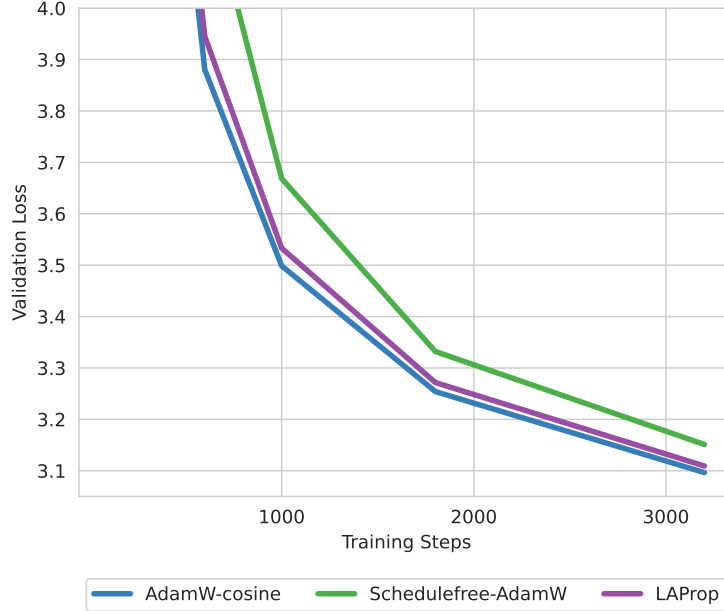


Figure 2: Best-run comparison of AdamW with cosine decay, Schedule-Free AdamW, and LARProp at higher batch size. Experimental details are provided in Section 5.1.

tailed weight averaging in this small-batch setting. AdEMAMix and Simplified-AdEMAMix perform best overall, which is consistent with the view that their updates align more closely with accelerated SGD variants.

5.1 LARGE BATCH SIZE EXPERIMENTS

The preceding experiments target a small-batch, noisy-gradient regime. We next evaluate the same optimizer ideas at a larger batch size to test whether the observed behavior persists when stochastic noise is reduced. In this setup, models are trained with a batch size of 1 million tokens over 3 billion tokens ($\approx$ Chinchilla scale).

Schedule-Free AdamW: As shown in Figure 2, Schedule-Free AdamW performs substantially worse than AdamW in this large-batch configuration. We attribute the gap to the coupling between weight averaging and momentum coefficients. At higher batch sizes, the preferred momentum value is much lower than $1 - 1/t$. Although a scaling factor $\approx 1 - r/t$ with $r \geq 1$ can reduce the effective momentum, increasing r also shortens the effective weight averaging window.

AdamW and Schedule-Free AdamW also differ in the ordering of momentum and preconditioning. AdamW applies momentum before preconditioning, whereas Schedule-Free AdamW applies preconditioning before momentum, making it algorithmically closer to LARProp (Ziyin et al., 2021). However, Figure 2 shows that AdamW and LARProp perform comparably, suggesting that this ordering difference is not the dominant source of the observed Schedule-Free AdamW gap.

AdEMAMix: In the large-batch regime, and consistent with Pagliardini et al. (2024), Figure 3 shows that setting $\beta_1 = 0.0$ in AdEMAMix causes a substantial drop relative to the full two-momentum version. This behavior is consistent with the accelerated SGD theory: AdEMAMix assigns a fixed weight of 1 to the current gradient, whereas theoretical accelerated SGD variants (Gupta et al., 2023) require the current-gradient weight to decrease as batch size increases.

Figure 3 also shows that Simplified-AdEMAMix matches AdEMAMix in this setting while removing the second momentum term. The best configuration achieves this at $\alpha = 0.0$, so Simplified-AdEMAMix reduces to standard Adam with momentum scheduling.

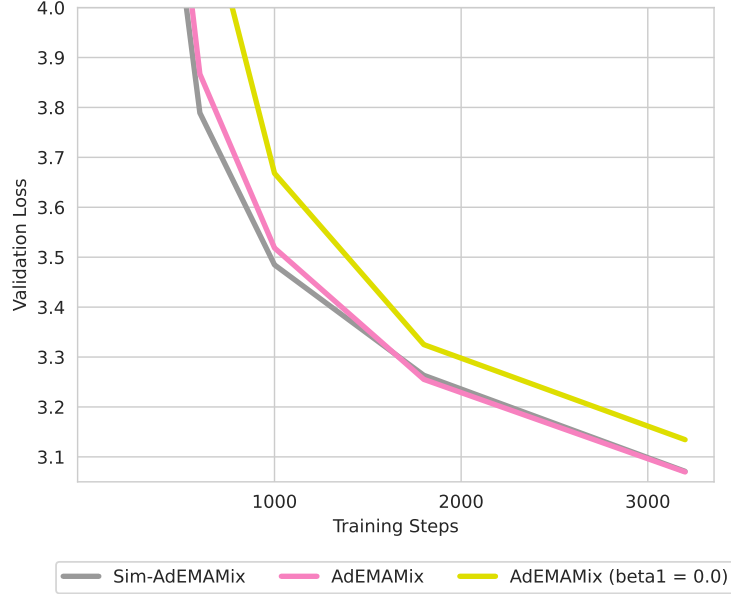


Figure 3: Best-run comparison of AdEMAMix, AdEMAMix with $\beta_1 = 0.0$, and simplified AdEMAMix in the higher-batch-size experiments. Experimental details are provided in Section 5.1.

6 CONCLUSION

This work connects accelerated SGD variants with several recently proposed optimizers, including Schedule-Free optimizers, AdEMAMix, MARS, and Lion. The analysis suggests that a broad set of practical momentum modifications can be understood as accelerated-SGD-style updates combined with preconditioning, averaging, or coordinate-wise transformations. In the small-batch language modeling experiments, AdEMAMix, the tested optimizer most closely aligned with the theoretical accelerated SGD form, achieves the strongest performance.

Motivated by this interpretation, we introduce Simplified-AdEMAMix, which removes the need for two momentum buffers. Across the small- and large-batch settings evaluated here, Simplified-AdEMAMix matches AdEMAMix while avoiding the additional memory overhead of the original method.

REFERENCES

- Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C., Lu, Y., and Le, Q. V. Symbolic discovery of optimization algorithms. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.
- Defazio, A. Momentum via primal averaging: Theoretical insights and learning rate schedules for non-convex optimization, 2021. URL <https://arxiv.org/abs/2010.00406>.
- Defazio, A., Yang, X., Mehta, H., Mishchenko, K., Khaled, A., and Cutkosky, A. The road less scheduled. *CoRR*, abs/2405.15682, 2024. doi: 10.48550/ARXIV.2405.15682. URL <https://doi.org/10.48550/arXiv.2405.15682>.
- Gupta, K., Siegel, J., and Wojtowysch, S. Achieving acceleration despite very noisy gradients, 2023. URL <https://arxiv.org/abs/2302.05515>.

- Jain, P., Kakade, S. M., Kidambi, R., Netrapalli, P., and Sidford, A. Accelerating stochastic gradient descent for least squares regression. In Bubeck, S., Perchet, V., and Rigollet, P. (eds.), *Proceedings of the 31st Conference On Learning Theory*, volume 75 of *Proceedings of Machine Learning Research*, pp. 545–604. PMLR, 06–09 Jul 2018. URL <https://proceedings.mlr.press/v75/jain18a.html>.
- Kidambi, R., Netrapalli, P., Jain, P., and Kakade, S. M. On the insufficiency of existing momentum schemes for stochastic optimization. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=rJTutzbA->.
- Liu, C. and Belkin, M. Accelerating sgd with momentum for over-parameterized learning. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rlgixp4FPH>.
- Lucas, J., Sun, S., Zemel, R., and Grosse, R. Aggregated momentum: Stability through passive damping. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Syxt5oC5YQ>.
- Ma, J. and Yarats, D. Quasi-hyperbolic momentum and adam for deep learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=SlfUpoR5FQ>.
- Pagliardini, M., Ablin, P., and Grangier, D. The ademamix optimizer: Better, faster, older. 2024. URL <https://arxiv.org/abs/2409.03137>.
- Porian, T., Wortsman, M., Jitsev, J., Schmidt, L., and Carmon, Y. Resolving discrepancies in compute-optimal scaling of language models. *CoRR*, abs/2406.19146, 2024. doi: 10.48550/ARXIV.2406.19146. URL <https://doi.org/10.48550/arXiv.2406.19146>.
- Vaswani, S., Bach, F., and Schmidt, M. Fast and faster convergence of sgd for over-parameterized models and an accelerated perceptron. In Chaudhuri, K. and Sugiyama, M. (eds.), *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pp. 1195–1204. PMLR, 16–18 Apr 2019. URL <https://proceedings.mlr.press/v89/vaswani19a.html>.
- Yuan, H., Liu, Y., Wu, S., Zhou, X., and Gu, Q. Mars: Unleashing the power of variance reduction for training large models, 2024. URL <https://arxiv.org/abs/2411.10438>.
- Zi Yin, L., Wang, Z. T., and Ueda, M. Laprop: Separating momentum and adaptivity in adam, 2021. URL <https://arxiv.org/abs/2002.04839>.

A HYPERPARAMETERS

This appendix reports the hyperparameter grids used for the small-batch experiments, keeping the comparison conditions fixed across optimizer families.

1. AdamW with cosine decay and 51.2k warmup: learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.99, 0.999, 0.99968, 0.9999]$. The best values were $\beta_1 = .9$ and $\beta_2 = .999$, matching common defaults. Although larger-batch settings often use $\beta_2 = .95$, improved behavior from higher β_2 at smaller batch sizes has also been observed by Porian et al. (2024).
2. AdamW with cosine decay and 10k warmup: learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$. Here β_1, β_2 are fixed to the best values from the previous sweep, and this shorter warmup underperforms the 51.2k-step warmup.
3. AdamW with constant-fraction weight averaging: learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, $\beta_1 = 0.9$, β_2 in $[0.99, 0.997, 0.999, 0.9997]$, δ in $[0.05, 0.1, 0.2]$.
4. AdamW with cosine decay and weight averaging: learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, δ in $[0.025, 0.05, 0.1]$.
5. Accelerated-SGD-based AdamW with cosine decay: learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, β_1 in $[0.999, 0.99968, 0.9999]$, β_2 in $[0.99, 0.9968, 0.999]$, $\beta_3 = 0.9$
6. Accelerated-SGD-based AdamW with constant learning rate and weight averaging: learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, β_1 in $[0.99684, 0.999]$, β_2 in $[0.999]$, $\beta_3 = 0.9$, δ in $[0.05, 0.1]$
7. Accelerated-SGD-based AdamW with cosine decay and weight averaging: learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, β_1 in $[0.99684, 0.999]$, $\beta_2 = 0.999$, δ in $[0.05, 0.1]$, $\beta_3 = 0.9$
8. Schedule-Free AdamW with constant learning rate: learning rate in $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3, 1\text{e-}2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
9. Schedule-Free AdamW with cosine decay: learning rates $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3, 1\text{e-}2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
10. MARS: learning rates $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3, 1\text{e-}2]$, β_1 in $[0.9, 0.95, 0.99]$, β_2 in $[0.99, 0.999]$, γ in $[0.0, 0.01, 0.02, 0.03, 0.04, 0.05]$, with precondition 1d set to True.
11. AdEMAMix: learning rates $[3.16\text{e-}4, 1\text{e-}3, 3.16\text{e-}3]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.999$, β_3 in $[0.99, 0.999, 0.9999]$, α in $[2, 4, 8, 16]$.
12. Sim-AdEMAMix: learning rates $[1\text{e-}6, 3.16\text{e-}6, 1\text{e-}5, 3.16\text{e-}5]$, β_1 in $[0.99, 0.999, 0.9999]$, $\beta_2 = 0.999$, α in $[10, 20, 50, 100]$

The corresponding hyperparameter sweeps for the large-batch experiments are:

1. Schedule-Free AdamW: learning rates $[1\text{e-}3, 3.16\text{e-}3, 1\text{e-}2]$, β in $[0.8, 0.9, 0.95]$, β_2 in $[0.9, 0.95]$, r in $[0.0, 5.0, 9.0, 50.0]$
2. AdamW: learning rates $[1\text{e-}3, 3.16\text{e-}3, 1\text{e-}2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
3. LProp: learning rates $[1\text{e-}3, 3.16\text{e-}3, 1\text{e-}2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
4. AdEMAMix: learning rates $[1\text{e-}3, 3.16\text{e-}3, 1\text{e-}2]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.95$, β_3 in $[0.9, 0.95, 0.99]$, α in $[2, 4, 8, 16]$
5. Sim-AdEMAMix: learning rates $[1\text{e-}4, 3.16\text{e-}4, 1\text{e-}3]$, β_1 in $[0.9, 0.95, 0.99]$, $\beta_2 = 0.95$, α in $[0.0, 0.5, 1.0]$

B EQUIVALENCE OF PREVIOUS ACCELERATION METHODS

Equation (1) gives the general accelerated SGD form used throughout the paper. This appendix shows that the methods of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023) all fall within that common form.

B.1 AGNES

The update from Gupta et al. (2023) is:

$$x'_n = x_n + \alpha v_n \quad x_{n+1} = x'_n - \eta g'_n \quad v_{n+1} = \rho_n(v_n - g'_n)$$

where g'_n is the stochastic gradient evaluated at x'_n , and the final function is evaluated at x_n . These equations can be rewritten as

$$x'_{n+1} = x'_n - \eta g'_n + \alpha v_{n+1} \quad -\frac{v_{n+1}}{\rho_n} = \rho_{n-1} \left(-\frac{v_n}{\rho_{n-1}} \right) + g'_n$$

Thus, x'_{n+1} follows an update of the form in Equation (1).

B.2 ASGD

The update from Jain et al. (2018) is:

$$y_{j-1} = \alpha x_{j-1} + (1 - \alpha)v_{j-1} \quad x_j = y_{j-1} - \delta g_{j-1} \quad z_{j-1} = \beta y_{j-1} + (1 - \beta)v_{j-1} \quad v_j = z_{j-1} - \gamma g_{j-1}$$

where g_{j-1} is the stochastic gradient evaluated at y_{j-1} , and the function is evaluated on the tail-averaged x sequence.

These updates can be rewritten as:

$$y_j = y_{j-1} - \alpha \delta g_{j-1} - (1 - \alpha)[y_{j-1} - v_j] \quad \frac{y_{j-1} - v_j}{\gamma - (1 - \beta)\alpha\delta} = (1 - \beta)\alpha \frac{y_{j-2} - v_{j-1}}{\gamma - (1 - \beta)\alpha\delta} + g_{j-1}$$

The resulting update follows the form of Equation (1).

B.3 MASS

The update from Liu & Belkin (2020) is:

$$w_{t+1} = u_t - \eta_1 g_t \quad u_{t+1} = (1 + \gamma)w_{t+1} - \gamma w_t + \eta_2 g_t$$

where g_t is the stochastic gradient evaluated at u_t , and the function is evaluated at w_t . These equations can be rewritten as:

$$u_{t+1} = u_t - \gamma(w_t - u_t) - [\eta_1(1 + \gamma) - \eta_2]g_t \quad \frac{w_t - u_t}{\eta_1\gamma - \eta_2} = \gamma \frac{w_{t-1} - u_{t-1}}{\eta_1\gamma - \eta_2} + g_t$$

The resulting update again fits the form of Equation (1).

B.4 SGD WITH NESTEROV ACCELERATION

The update from Vaswani et al. (2019) is:

$$w_{k+1} = \zeta_k - \eta g_k \quad \zeta_k = \alpha_k v_k + (1 - \alpha_k)w_k \quad v_{k+1} = \beta_k v_k + (1 - \beta_k)\zeta_k - \gamma_k \eta g_k$$

These equations can be rewritten as:

$$\zeta_{k+1} = \zeta_k - \eta g_k + \alpha_{k+1}[v_{k+1} - w_{k+1}]; \quad v_{k+1} - w_{k+1} = \beta_k(1 - \alpha_k)[v_k - w_k] - \eta(\gamma_k - 1)g_k$$